

Machine Learning Evaluation

Metrics that matter: precision, recall, F1, ROC-AUC, cross-validation, and learning curves

1. Why Evaluation Matters

A model is only as good as our measurement of it. Careless evaluation is worse than no evaluation at all, because it manufactures false confidence: a model that scores 98% on a leaked test set can fail catastrophically on real users. Rigorous evaluation has two pillars — an honest data split and the right metric for the job.

The honest split, introduced in *Introduction to Machine Learning*, keeps a test set completely quarantined until final reporting; the validation set guides every intermediate decision. This document focuses on the second pillar: choosing and interpreting metrics, validating them robustly, and using evaluation curves to diagnose problems like overfitting.

2. The Confusion Matrix

For binary classification, every prediction falls into one of four buckets, conventionally arranged in a two-by-two *confusion matrix*.

- **True positives (TP):** predicted positive, actually positive.
- **True negatives (TN):** predicted negative, actually negative.
- **False positives (FP):** predicted positive, actually negative — a false alarm.
- **False negatives (FN):** predicted negative, actually positive — a missed case.

Nearly every classification metric is arithmetic on these four counts. Accuracy, for example, is $(TP + TN) / (TP + TN + FP + FN)$: the fraction of all predictions that were correct. As noted in *Introduction to Machine Learning*, accuracy collapses as a useful metric when classes are imbalanced, which motivates the finer-grained metrics below.

3. Precision and Recall

Precision and *recall* split the notion of correctness into two complementary questions.

- **Precision = $TP / (TP + FP)$.** Of everything the model *flagged* as positive, how many truly were? Precision punishes false alarms.
- **Recall = $TP / (TP + FN)$.** Of everything *actually* positive, how many did the model catch? Recall punishes misses.

The two are typically in tension: lowering the decision threshold catches more positives (higher recall) but also sweeps in more false alarms (lower precision). Neither dominates the other in general — the right balance depends on the cost of each error type. In spam filtering, low precision annoys users (legitimate mail hidden); in cancer screening, low recall is far worse (a missed tumor). A good answer to 'what is the difference between precision and recall' always names both the formulas and this cost-driven tradeoff.

Precision and recall are computed at a single decision threshold. Sweeping the threshold traces out the *precision-recall curve*; the area under it summarizes the tradeoff in one number and is especially informative for imbalanced datasets.

4. The F1 Score

When a single number is needed that respects both precision and recall, the *F1 score* is the standard choice. It is the *harmonic mean* of precision (P) and recall (R): $F1 = 2PR / (P + R)$.

The harmonic mean, unlike the arithmetic mean, punishes extreme imbalance: a model with precision 1.0 and recall 0.0 scores $F1 = 0$, not 0.5. This makes F1 honest in exactly the situations where accuracy lies — imbalanced classes, rare events, information retrieval. Its limitation is that it weights precision and recall equally; when the costs are asymmetric, practitioners use the weighted F-beta variant or simply report the pair.

5. Accuracy and Its Limits

Accuracy remains the right metric when classes are balanced and all errors cost about the same — for instance, classifying balanced product-review sentiment. Its failure modes are worth memorizing: on a dataset with 99% negatives, a do-nothing classifier that always predicts negative scores 99% accuracy while detecting nothing.

Two habits protect against accuracy theater. First, always compare against a *baseline*: the majority-class classifier and, when available, a simple heuristic. A model is only interesting if it beats the trivial alternative. Second, report accuracy alongside precision, recall, and F1 rather than alone, so readers can see what the single number hides.

6. ROC Curves and AUC

The *receiver operating characteristic* (ROC) curve plots the true positive rate (recall) against the false positive rate ($FP / (FP + TN)$) as the decision threshold varies. A perfect classifier hugs the top-left corner; a random classifier follows the diagonal.

The *area under the ROC curve* (ROC-AUC) compresses this into one number between 0 and 1: 1.0 is perfect, 0.5 is random guessing. AUC has a useful probabilistic interpretation — it is the probability that a randomly chosen positive example is ranked above a randomly chosen negative one. It is threshold-independent, which makes it ideal for comparing models before a deployment threshold is chosen. Its caveat: on heavily imbalanced data, ROC-AUC can look rosy while precision is poor, so pair it with the precision-recall curve in those settings.

7. Cross-Validation

A single train/validation split can be lucky or unlucky, especially on small datasets. *K-fold cross-validation* addresses this by partitioning the data into K equal folds, training on K-1 folds and validating on the remaining one, and rotating so each fold serves as validation exactly once. The K scores are averaged, yielding a more stable estimate, and their spread quantifies uncertainty.

The price is computation — K full training runs — so $K = 5$ or 10 is the usual compromise. *Stratified* K -fold preserves the class ratio in every fold and is the default for classification. One non-negotiable rule: all preprocessing that learns from data (scaling, imputation, feature selection) must happen *inside* each fold, or information leaks from validation into training and the estimates turn optimistic.

8. Detecting Overfitting with Learning Curves

Learning curves plot training and validation error against training-set size (or, equivalently, against training iterations). They are the single most informative diagnostic in applied machine learning.

The healthy pattern is convergence: both curves descend and meet at a low error, leaving only a small generalization gap. The *overfitting* signature is divergence: training error keeps falling toward zero while validation error flattens or rises — the model is memorizing rather than learning. The *underfitting* signature is a high plateau: both curves stall at poor performance, indicating the model family lacks the capacity for the task.

Each signature prescribes its remedy. Divergence calls for more data, regularization, simpler models, or early stopping — the toolkit from *Introduction to Machine Learning* and the deep-learning variants (dropout, weight decay) from *Neural Networks and Deep Learning*. A high plateau calls for more expressive models, better features, or less aggressive regularization. Learning curves thus close the loop: they turn the abstract bias-variance tradeoff into a concrete, visual debugging procedure.